

DAY 5 — COMBINATIONAL BUILDING BLOCKS

Context Acknowledged

Days 1–4 complete. You can write synthesizable flip-flops, registers, counters, and FSMs. That is a solid sequential logic foundation. Now we shift to **combinational building blocks** — the circuits that do actual data computation. These feed directly into the ALU project.

Days 5–10 Roadmap (Brief)

Day	Topic	What You Build
5	Combinational Blocks	MUX, Decoder, Encoder, Comparator, Adder
6	ALU Architecture	ADD, SUB with flags; Logic unit
7	ALU Completion	Shift unit, full ALU integration
8	Testbench	Self-checking, comprehensive coverage
9	Synthesis	Yosys clean RTL, no latches
10	Documentation	README, block diagram, portfolio delivery

Every piece you build in Day 5 becomes a **sub-module or design pattern inside the ALU**. Nothing is throwaway work.

DAY 5 — SESSION 1: MULTIPLEXERS

BLOCK 1: CONCEPT (10–12 min)

What is a Multiplexer?

A **multiplexer (MUX)** is a **data selector**. It has:

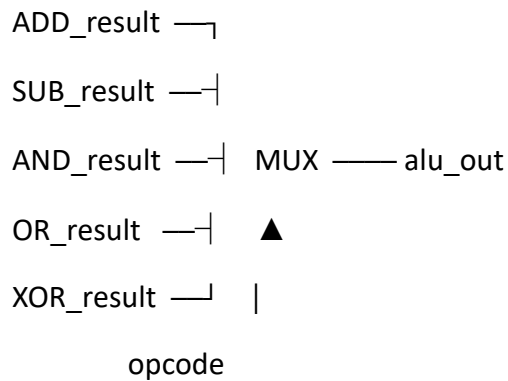
- **N data inputs**
- **1 output**
- **Select lines** that choose which input appears at the output

Nothing is computed. Nothing is stored. The output is **a direct wire to one of the inputs**, chosen by the select signal.

Why Does This Matter for an ALU?

Inside your ALU, after the arithmetic unit computes ADD, and the logic unit computes AND, and the shift unit computes SHL — **how does the ALU choose which result to output?**

A **MUX**. The opcode signal drives the select lines, and the final MUX picks the correct result.



This is the **core output-selection pattern** in every ALU ever built.

MUX Sizing

Name	Data Inputs	Select Lines	Formula
2:1 MUX	2	1	$2^1 = 2$
4:1 MUX	4	2	$2^2 = 4$
8:1 MUX	8	3	$2^3 = 8$
16:1 MUX	16	4	$2^4 = 16$

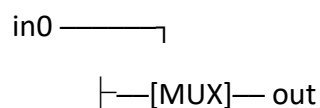
Rule: **N select lines → 2^N data inputs.**

2:1 MUX Truth Table

sel out

0 in0

1 in1



in1 ———┘



sel

When sel = 0: out = in0 When sel = 1: out = in1

4:1 MUX Truth Table

sel[1] sel[0] out

0 0 in0

0 1 in1

1 0 in2

1 1 in3

Hardware Reality

A MUX synthesizes to:

- **Transmission gates** in CMOS
- **LUT (Look-Up Table)** in FPGA
- **Standard cell mux** in ASIC

A 2:1 MUX is one of the **most fundamental cells** in any standard cell library. Everything from clock gating to data path selection uses MUXes.

Common Mistakes (Before You Make Them)

Mistake 1: Using if-else in always @(*) without covering all cases → LATCH

// WRONG — creates latch if sel is not 0 or 1

always @(*) begin

 if (sel == 0) out = in0;

 // forgot else → latch!

end

Mistake 2: Forgetting that MUX is purely combinational No clock. No reset. No posedge. If you write always @(posedge clk) for a MUX, you have built a **register**, not a MUX.

Mistake 3: Bit-width mismatch If in0 is 8 bits and out is 4 bits, Verilog silently truncates. You lose data with no error.

BLOCK 2: CODE & PRACTICAL (10–12 min)

We build three MUXes. Each one teaches something new.

Design 1: 2:1 MUX — Three Legal Ways

```
// FILE: mux2to1.v

// 2:1 Multiplexer — Three implementation styles
// Synthesizable RTL — Verilog-2001

// =====
// METHOD 1: Continuous assign with ternary operator
// This is the preferred, cleanest RTL style for simple MUXes
// =====

module mux2to1_ternary #(
    parameter WIDTH = 8
)(
    input wire [WIDTH-1:0] in0, // Input 0 (selected when sel=0)
    input wire [WIDTH-1:0] in1, // Input 1 (selected when sel=1)
    input wire          sel, // Select line
    output wire [WIDTH-1:0] out // Output
);

// Ternary: (condition) ? value_if_true : value_if_false
// sel=1 → out=in1, sel=0 → out=in0
assign out = sel ? in1 : in0;
```

```
endmodule
```

```
// =====
```

```
// METHOD 2: Continuous assign with bitwise operations
```

```
// Matches the actual gate-level logic of a MUX
```

```
// out = (in1 & sel) | (in0 & ~sel)
```

```
// =====
```

```
module mux2to1_gates #(
```

```
    parameter WIDTH = 8
```

```
)(
```

```
    input wire [WIDTH-1:0] in0,
```

```
    input wire [WIDTH-1:0] in1,
```

```
    input wire      sel,
```

```
    output wire [WIDTH-1:0] out
```

```
);
```

```
    // This is exactly what a textbook MUX gate equation looks like
```

```
    // Each bit of out is: (in1_bit AND sel) OR (in0_bit AND NOT sel)
```

```
    // sel is 1-bit, but Verilog replicates it across all WIDTH bits
```

```
    assign out = (in1 & {WIDTH{sel}}) | (in0 & {WIDTH{~sel}});
```

```
endmodule
```

```
// =====
```

```
// METHOD 3: always @(*) with case statement
```

```
// Use this pattern for 4:1 and larger — scales better
```

```
// =====
```

```

module mux2to1_case #(
    parameter WIDTH = 8
)(
    input wire [WIDTH-1:0] in0,
    input wire [WIDTH-1:0] in1,
    input wire      sel,
    output reg [WIDTH-1:0] out // NOTE: reg because driven by always block
);

always @(*) begin
    case (sel)
        1'b0: out = in0;
        1'b1: out = in1;

        // default not needed for 1-bit sel (0 and 1 cover all cases)
        // but add it for safety in larger designs
        default: out = {WIDTH{1'b0}};
    endcase
end

endmodule

```

Key point on reg vs wire:

- assign statement → output must be wire
- always block → output must be reg
- This is a Verilog rule, not a hardware choice. Both synthesize to the same combinational logic.

Design 2: 4:1 MUX

// FILE: mux4to1.v

// 4:1 Multiplexer

// sel[1:0] selects one of four inputs

```
module mux4to1 #(
    parameter WIDTH = 8
)(
    input wire [WIDTH-1:0] in0,
    input wire [WIDTH-1:0] in1,
    input wire [WIDTH-1:0] in2,
    input wire [WIDTH-1:0] in3,
    input wire [1:0] sel, // 2-bit select: covers 4 choices
    output reg [WIDTH-1:0] out
);

always @(*) begin
    case (sel)
        2'b00: out = in0;
        2'b01: out = in1;
        2'b10: out = in2;
        2'b11: out = in3;
        default: out = {WIDTH{1'b0}}; // Defensive default
    endcase
end

endmodule
```

Why default even when all 4 cases are covered?

sel is 2 bits → 4 possible values → you covered all 4. Technically safe. But tools like Yosys and linting tools are happier with explicit default. It also protects you if you later change sel width. **Always write default in case statements.**

```

// FILE: mux8to1.v

// 8:1 Multiplexer — notice the pattern scaling from 4:1

module mux8to1 #(
    parameter WIDTH = 8
)(
    input wire [WIDTH-1:0] in0, in1, in2, in3,
    input wire [WIDTH-1:0] in4, in5, in6, in7,
    input wire [2:0] sel, // 3-bit select: covers 8 choices
    output reg [WIDTH-1:0] out
);

always @(*) begin
    case (sel)
        3'b000: out = in0;
        3'b001: out = in1;
        3'b010: out = in2;
        3'b011: out = in3;
        3'b100: out = in4;
        3'b101: out = in5;
        3'b110: out = in6;
        3'b111: out = in7;
        default: out = {WIDTH{1'b0}};
    endcase
end

endmodule

```

Testbench for 4:1 MUX

```

// FILE: tb_mux4to1.v

// Testbench for 4:1 MUX — self-checking


`timescale 1ns/1ps


module tb_mux4to1;


    // ---- Parameters ----
    parameter WIDTH = 8;


    // ---- DUT Signals ----
    reg [WIDTH-1:0] in0, in1, in2, in3;
    reg [1:0]      sel;
    wire [WIDTH-1:0] out;


    // ---- Instantiate DUT ----
    mux4to1 #(.WIDTH(WIDTH)) dut (
        .in0(in0), .in1(in1),
        .in2(in2), .in3(in3),
        .sel(sel),
        .out(out)
    );


    // ---- Task: Check result ----
    integer pass_count, fail_count;


    task check;
        input [WIDTH-1:0] expected;

```

```

input [63:0]    test_num;

begin

    #1; // Let combinational logic settle

    if (out === expected) begin

        $display("PASS [Test %0d] sel=%b out=%0d (expected %0d)",
            test_num, sel, out, expected);

        pass_count = pass_count + 1;

    end else begin

        $display("FAIL [Test %0d] sel=%b out=%0d (expected %0d)",
            test_num, sel, out, expected);

        fail_count = fail_count + 1;

    end

end

endtask

```

```

// ---- Stimulus ----

```

```

initial begin

```

```

    pass_count = 0;

```

```

    fail_count = 0;

```

```

    // Set fixed input values

```

```

    in0 = 8'd10;

```

```

    in1 = 8'd20;

```

```

    in2 = 8'd30;

```

```

    in3 = 8'd40;

```

```

    // Test each select value

```

```

    sel = 2'b00; check(in0, 1);

```

```

sel = 2'b01; check(in1, 2);

sel = 2'b10; check(in2, 3);

sel = 2'b11; check(in3, 4);


// Change inputs, re-verify (MUX must track input changes)

in1 = 8'd99;

sel = 2'b01; check(8'd99, 5);


// Boundary values

in0 = 8'd0;

in3 = 8'd255;

sel = 2'b00; check(8'd0, 6);

sel = 2'b11; check(8'd255, 7);


// ---- Summary ----

$display("-----");

$display("TOTAL: %0d passed, %0d failed", pass_count, fail_count);

if (fail_count == 0)

    $display("ALL TESTS PASSED");

else

    $display("FAILURES DETECTED");

$display("-----");


$finish;

end


// ---- Waveform dump ----

initial begin

```

```
$dumpfile("mux4to1.vcd");  
$dumpvars(0, tb_mux4to1);  
end
```

```
endmodule
```

Running It

Compile

```
iverilog -o mux4to1_sim tb_mux4to1.v mux4to1.v
```

Run

```
./mux4to1_sim
```

View waveform (optional)

```
gtkwave mux4to1.vcd
```

Expected output:

```
PASS [Test 1] sel=00 out=10 (expected 10)  
PASS [Test 2] sel=01 out=20 (expected 20)  
PASS [Test 3] sel=10 out=30 (expected 30)  
PASS [Test 4] sel=11 out=40 (expected 40)  
PASS [Test 5] sel=01 out=99 (expected 99)  
PASS [Test 6] sel=00 out=0 (expected 0)  
PASS [Test 7] sel=11 out=255 (expected 255)
```

```
-----
```

TOTAL: 7 passed, 0 failed

ALL TESTS PASSED

```
-----
```

BLOCK 3: MINI TASK (5–8 min)

Your Task

Write a **parameterized 2:1 MUX** with these exact requirements:

1. Module name: mux2to1_task
2. Parameter: WIDTH defaulting to 4
3. Inputs: a, b (WIDTH-bit each), sel (1-bit)
4. Output: y (WIDTH-bit)
5. Use assign with ternary operator
6. Write a small testbench that tests at least **4 cases** and prints PASS/FAIL

Constraints:

- No always block — use assign only
- Output must be wire
- Testbench must be self-checking (not just \$display of values — it must compare and print PASS or FAIL)

Post your code. I will review every line.

While you work: Think about this — in the ALU, when opcode = 3'b000 means ADD and opcode = 3'b001 means SUB, what kind of MUX will select the final output? How many inputs will it need? How many select bits? You will build that exact circuit in Day 6.